TatvaSoft

SOFTWARE DEVELOPMENT · GUIDELINE

TVS-DEV-AI-GL-01.1

Prompts — Do’s and Don’ts

Practical Do’s and Don’ts for writing effective prompts to help developers work with AI editors more efficiently, consistently, and with better control over the generated output.

VERSION	EFFECTIVE DATE	PREPARED BY	APPROVED BY
1.0	28.09.2026	Vishal Shah	Shital Patel

HOW TO READ THIS DOCUMENT

Short on time? Read the one-page **Prompting at a Glance** — it lists every Do and Don’t.

Each Do and Don’t has its own section, with **Example Prompts** shown in grey boxes.

Parent guideline: **TVS-DEV-AI-GL-01 AI Development Tools Guidelines** (Prompting section).

Document Control

Document Control Information

Document Title	Prompts — Do’s and Don’ts
Document ID	TVS-DEV-AI-GL-01.1
Document Version	Version 1.0
Effective Date	28.09.2026
Parent Document	TVS-DEV-AI-GL-01 — AI Development Tools Guidelines

Document Publication History

Document Prepared By	Vishal Shah
Document Reviewed By	Parthesh Mehta
Document Approved By	Shital Patel
Initial Release Version	1.0
Initial Release Date	28.09.2026

Document Revision History

Version No.	Revision Description	Change Author	Change Approver	Revision Date
1.0	Final Release	Vishal Shah	Shital Patel	25.09.2026

Document Definition

This document outlines some practical Do’s and Don’ts for writing effective prompts to help developers work with AI editors more efficiently, consistently, and with better control over the generated output.

Contents

Prompting at a Glance	Every Do and Don’t on one page
Introduction	Why prompt quality matters
Prompting Do’s	11 practices to follow
Prompting Don’ts	6 practices to avoid

Prompting at a Glance

Every Do and Don’t in this document on one page. Select an item to jump to its details.

✓ PROMPTING DO’S

- ✓ Use a structured template for prompt design
- ✓ Define the scope
- ✓ Provide relevant context
- ✓ Ask the AI to inspect before modifying (Complex Tasks)
- ✓ Ask for multiple approaches when the solution isn't obvious
- ✓ Work in smaller chunks
- ✓ Define acceptance criteria
- ✓ Ask the AI to explain risky changes
- ✓ Review the difference after every meaningful Agent change
- ✓ Specify the required skill when the task needs a specific capability
- ✓ Optimize for context length and token usage

✗ PROMPTING DON’TS

- ✗ Use vague prompts
- ✗ Ask AI to modify code before understanding the problem
- ✗ Accept the first solution automatically
- ✗ Let AI rewrite large portions of working code unnecessarily
- ✗ Allow unrelated refactoring during feature development
- ✗ Pass confidential data or secrets in prompts

Introduction

> **KEY TAKEAWAY**
>
> When working with AI-powered editors such as Cursor, Antigravity, Claude and similar tools, **the quality of the prompt plays an important role in the quality and relevance of the output.**

✓ CLEAR, WELL-STRUCTURED PROMPT

A clear and well-structured prompt helps the AI understand the expected **scope, context, constraints, and desired outcome**.

✗ VAGUE OR OVERLY BROAD PROMPT

On the other hand, vague or overly broad prompts can lead to **unnecessary code changes, incorrect assumptions, or additional rework**.

This document outlines some practical **Do’s and Don’ts for writing effective prompts** to help developers work with AI editors more efficiently, consistently, and with better control over the generated output.

Prompting Do’s

DO: Use a structured template for prompt design

For major tasks, format your prompt using clear operational parameters:

Parameter	Example
Goal	Add pagination to the customer search API.
Context	Search currently loads all records at once, causing timeouts on large datasets.
Scope	Modify only `customer_service.py` and `customer_repository.py`. Do not modify API schemas (Pydantic/FastAPI) or database models/migrations.
Rules	Default to 20 records per page; support `page` and `page_size` query parameters.
Done when	Unit tests pass for single-page and multi-page queries, and invalid page indices return a 400 Bad Request.
Output	Return only the modified code methods.

DO: Define the scope

Explicitly tell the AI:

- ☐ Which files/modules it can modify
- ☐ Which files it should not modify
- ☐ Whether database changes are allowed
- ☐ Whether APIs/contracts can change
- ☐ Whether new dependencies can be introduced

> **EXAMPLE**
> Modify only the customer service and repository layers. Do not change the API response contract or introduce new dependencies.

DO: Provide relevant context

Include:

- ☐ Business requirement
- ☐ Existing behavior
- ☐ Expected behavior
- ☐ Technical constraints
- ☐ Relevant error/log
- ☐ Acceptance criteria
- ☐ Related files/modules

> **NOTE**
> Don't expect the AI to understand **undocumented business rules**.

DO: Ask the AI to inspect before modifying (Complex Tasks)

For unfamiliar or existing/complex code:

01 First inspect the existing implementation and explain how the current flow works. **Do not make any changes yet.**

02 *Then, after reviewing the response:*
Proceed with the implementation based on the approach discussed.

→ *This creates a review checkpoint before code modification.*

DO: Ask for multiple approaches when the solution isn't obvious

For architectural or technical decisions:

EXAMPLE PROMPT

Analyze 2–3 possible approaches, including advantages, disadvantages, complexity, performance and maintainability. **Do not implement anything yet.**

→ *Then select the approach deliberately.*

DO: Work in smaller chunks

✗ **INSTEAD OF**

Implement the entire order management module.

✓ **BREAK IT DOWN**

1. Analyze existing order architecture
2. Design database changes
3. Implement repository layer
4. Implement service layer
5. Implement APIs
6. Add validation
7. Add tests
8. Review the complete implementation

→ *Smaller tasks give developers better control over AI-generated changes.*

DO: Define acceptance criteria

EXAMPLE

The implementation is complete only when:

- ✓ Existing APIs continue to work
- ✓ Invalid customer IDs return 404
- ✓ Duplicate records are prevented
- ✓ Unit tests cover success and failure scenarios
- ✓ No unrelated files are modified

→ *This gives the AI a measurable target.*

DO: Ask the AI to explain risky changes

FOR EXAMPLE

Before modifying the database schema, explain the impact on existing data, migrations and dependent APIs.

DO: Review the difference after every meaningful Agent change

Developers should check:

- ☐ What files changed?
- ☐ What code was added?
- ☐ What code was removed?
- ☐ Were unrelated files modified?
- ☐ Were dependencies added?
- ☐ Did the AI alter existing behavior?
- ☐ Are error-handling and edge cases covered?

REMEMBER

Never treat "Agent completed successfully" as equivalent to "implementation is correct."

DO: Specify the required skill when the task needs a specific capability

Be explicit when you know a particular skill is required for the task:

EXAMPLE PROMPT

"Use the API documentation skill for this task and follow the guidelines defined within it."

→ *This helps the AI use the intended capability and follow the relevant instructions instead of relying on its own assumptions.*

DO: Optimize for context length and token usage

Limit context references	Explicitly tag or mention only the specific files needed (e.g., "Check only `@customer_service.py` and `@customer_repository.py`").
Restrict response size	Request minimal edits rather than full-file reprints (e.g., *"Reply with only the changed code snippet"*).

> **TIP — NEW TASK = NEW CHAT**
> Clear the chat history or open a **new chat session** when switching to a completely different task to avoid paying for unrelated context tokens.

Prompting Don’ts

DON'T: Use vague prompts

✗ AVOID

- ✗ *Fix this.*
- ✗ *Make it better.*
- ✗ *Optimize this.*
- ✗ *Implement the feature.*

✓ INSTEAD

Specify **what is wrong**, **what is expected**, and **what constraints exist**.

DON'T: Ask AI to modify code before understanding the problem

Especially with bugs.

✗ AVOID

Fix the login issue.

✓ BETTER

Analyse why users are intermittently receiving 401 responses during token refresh. Review the authentication flow and logs. Do not modify code yet. First identify the likely root cause and supporting evidence.

DON'T: Accept the first solution automatically

AI can produce a technically valid solution that is **not the best solution** for the application.

Developers should challenge:

- ☐ Why this approach?
- ☐ Are there simpler alternatives?
- ☐ What are the performance implications?
- ☐ Does it follow existing architecture?
- ☐ Does it introduce technical debt?
- ☐ Does it affect existing functionality?

DON'T: Let AI rewrite large portions of working code unnecessarily

A bug in one function does not justify rewriting the entire module.

Use instructions such as:

> EXAMPLE PROMPT
>
> Make the minimum required change to resolve the issue. Preserve all existing behavior.

DON'T: Allow unrelated refactoring during feature development

Be explicit:

> EXAMPLE PROMPT
>
> Do not refactor unrelated code or perform formatting changes outside the affected files.
>
> → *This significantly reduces unnecessary AI-generated changes.*

DON'T: Pass confidential data or secrets in prompts

CRITICAL

Never include **API keys, passwords, database connection strings, or PII** (personally identifiable information) in your prompts or code snippets.

— End of Document —